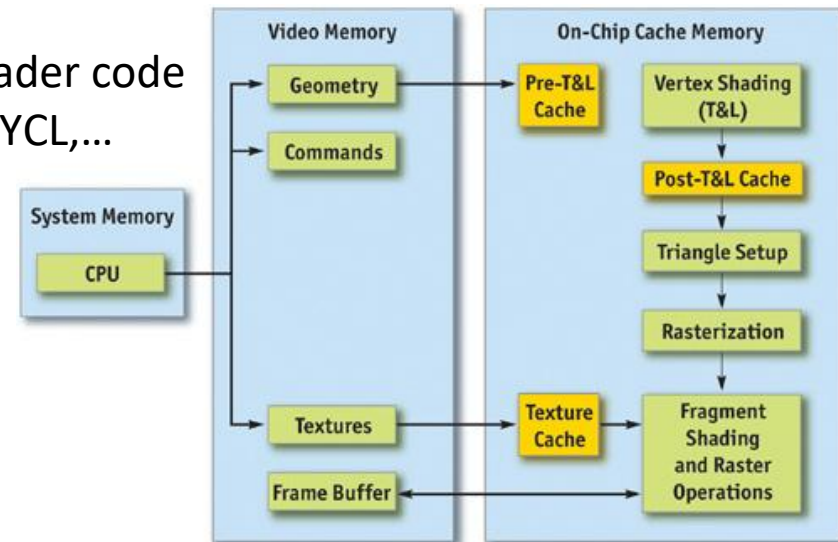


Parallel Programming on GPUs

DISTRIBUTED AND PARALLEL TRAINING

GPGPU - General Purpose Computing on GPUs

- Traditionally, GPUs are used for graphic rendering, i.e. transforming from 3D models to 2D pictures
 - 3D objects are represented as a set of 3D polygons on the surface of the objects
 - 3D polygons are transformed into 2D polygon using a graphic pipeline
 - They are shaded with customizable (programmable) shaders
 - Multiple polygons can be transformed in parallel
- That model of computation was generalized for general purpose computation → GPGPU
 - Same code (similar to a shader) can be executed on multiple of data objects in parallel
 - Old time: programmers hack into shader code
 - Today: CUDA, OpenCL, AMD ROCs, SYCL,...



NVIDIA GPGPU



- An NVIDIA branding for GPGPU cards
 - Specialized for general-purpose computation
 - Was used in many supercomputers and high-end HPC systems
 - Programmable using CUDA, OpenCL
 - Discontinued in May 2020(conflict with Tesla cars), but continued in another brand – NVIDIA Data Center GPUs

NVIDIA GPGPUs

Fermi



2010

Tsubame 2.0 (M2050)



GSIC/TITech
2.3 PFLOPs/s

Kepler



2012

Titan (k20x)



ORNL
17.6 PFLOPs/s
★ 3x #cores (1536)
★ improved power efficiency

Pascal



2016

Piz Daint (P100)



CSCS
25.4 PFLOPs/s
★ NVLink, 5x PCIe bw
★ HBM2, 3x memory bw
★ Unified memory, multi-GPU/CPU

Volta



2018

Summit/Sierra (V100)



ORNL/LLNL
148.6/94.6 PFLOPs/s
★ NVLink2, 2x previous
★ AI, 640 tensor cores

NVIDIA GPGPUs

Ampere



2020

Selene (A100)
NVIDIA
79.22 PFlops

Hopper



2022

Eagle (H100)
Microsoft
846.84 PFlops

Blackwell



2024

CUDA : Introduction

- Compute Uniform Device Architecture (2006)
- A parallel programming framework for programming NVIDIA GPGPUs
- Components
 - Driver: nvidia.ko, libcuda.so
 - CUDA Toolkit:
 - Compiler: nvcc
 - Debugger: cuda-gdb
 - Libcudart.so, libcublas.so,...
 - CUDA SDK
 - examples



CUDA : Introduction

- GPU programming model based on threads, share memory, barrier synchronization
- Linux, MacOS, Windows supported
- Extension of C/C++
- Support multiple GPUs per host
- SIMD, compute intensive, highly parallel computation
- Can interoperate with C, C++, Fortran, Python,...
- PTX – Forward compatible



CUDA : Programming

Single Thread Block Vector Addition

kernel declaration specifier

```
// kernel definition;
__global__ void VecAdd(float *A, float *B, float *C)
{
    int i;
    i = threadIdx.x;
    C[i] = A[i] + B[i];
}

int main()
{
    ...
    // kernel invocation with N threads
    VecAdd <<< 1, N >>> (A, B, C);
    ...
}
```

built-in variables,
e.g. threadIdx.x=0,1,2...

kernel execution configuration

<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

CUDA : Programming

```
[sh@n71-006]$ nvcc single_thread_block_vector_addition.cu
[sh@n71-006]$ ./a.out
```

```
0 100.000000
1 100.000000
...
99 100.000000
```

```
[sh@n71-006]$ watch nvidia-smi
```

```
Every 2.0s: nvidia-smi                      Wed Mar  4 15:04:17 2020
Wed Mar  4 15:04:17 2020
```

```
+-----+
| NVIDIA-SMI 430.14      Driver Version: 430.14      CUDA Version: 10.2      |
+-----+-----+-----+-----+-----+-----+
| GPU  Name            Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+=====+=====+=====+=====+
|   0   GeForce GTX 1080        Off | 00000000:41:00.0 Off |                     |
| 24%    39C    P2     73W / 180W |  241MiB /  8119MiB |      61%      Default |
+-----+-----+-----+-----+-----+-----+
+-----+
| Processes:                                     GPU Memory |
|  GPU       PID    Type    Process name      Usage      |
+-----+-----+-----+-----+-----+-----+
|      0      84817    C      ./a.out             103MiB |
+-----+-----+-----+-----+-----+-----+
+-----+
```

CUDA : Programming

- `__global__` : declares a function as being a GPU-kernel
 - executed on the device
 - callable from the host
 - also callable from the device for devices of compute capability ≥ 3.2
- `__host__` declares a function as being a host-kernel
 - executed on the host
 - callable from the host only
 - default when omitted
- `__device__` declares a function as being a GPU-only-kernel
 - executed on the device
 - callable from the device only

CUDA : Programming

- `threadIdx.[x,y,z]` may be one-dimensional, two-dimensional or three-dimensional referring to a one-dimensional, two-dimensional or three-dimensional thread block executed on the host
- `threadIdx.[x,y,z]` provides a direct formal abstraction of domains, ie facilitates straightforward reference to the elements of a vector, matrix, or a volume
- `threadIdx.[x,y,z]` for N threads goes from 0 to N-1
- when working with thread blocks of two/three-dimensional shapes, the declaration switches from `int N;` to `dim3 threadsPerBlock(N, N);` (structure with 3 members, x, y, z, of type int)

CUDA : Programming

Single Thread Block Matrix Addition

```
// kernel definition;
__global__ void MatAdd(float **A, float **B, float **C)
{
    int i, j;
    i = threadIdx.x;
    j = threadIdx.y;
    C[i][j] = A[i][j] + B[i][j];
}

int main()
{
    int numBlocks;
    dim3 threadsPerBlock;
    ...
    // kernel invocation with one block of N * N threads
    numBlocks = 1;
    threadsPerBlock.x = N;
    threadsPerBlock.y = N;
    MatAdd <<< numBlocks, threadsPerBlock >>> (A, B, C);
    ...
}
```

2D thread
block ini-
tialization

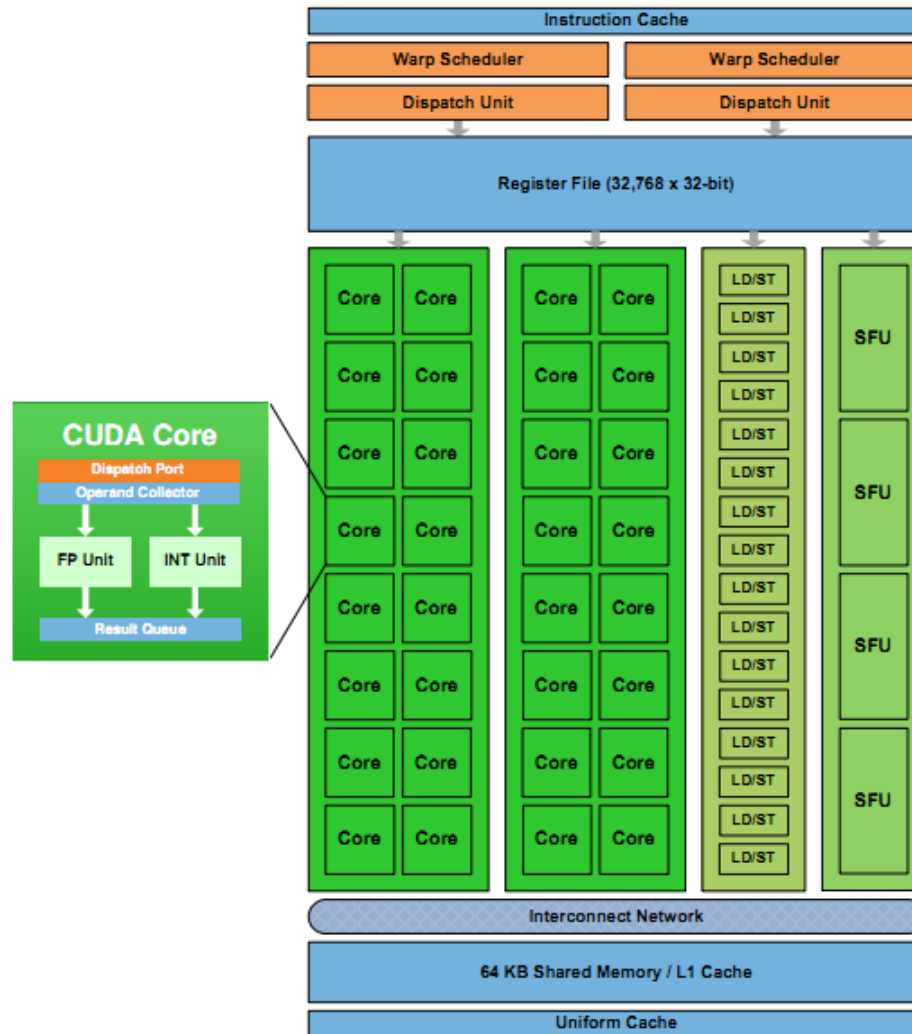
built-in
2D thread
indices
0,1,2...

goes di-
rectly into
kernel exe-
cution con-
figuration

CUDA : Programming

- There is an upper limit to the number of threads in a thread block, e.g. ≈ 1024 for current GPUs, because all threads of a thread block are supposed to run on the same SM (streaming multiprocessor)
- A single SM typically contains 64-128 CUDA cores
- Different GPU architectures vary in terms of numbers of SMs, e.g. V100 has 80 SMs, gtx1080 has 20 SMs
- However multiple thread blocks can be launched in parallel as defined by the initial parameter numBlocks used in the kernel execution configuration
`<<<numBlocks,threadsPerBlock>>>`

CUDA : Programming



Mỗi SM trong kiến trúc Fermi có 32 lõi xử lý vô hướng

CUDA : Programming

```
[sh@n71-006]$ deviceQuery
```

```
/NVIDIA_CUDA-9.1_Samples/1_Uutilities/deviceQuery/deviceQuery Starting...
```

```
CUDA Device Query (Runtime API) version (CUDART static linking)
```

```
Detected 1 CUDA Capable device(s)
```

```
Device 0: "GeForce GTX 1080"
```

CUDA Driver Version / Runtime Version	9.1 / 9.1
CUDA Capability Major/Minor version number:	6.1
Total amount of global memory:	8114 MBytes (8508145664 bytes)
<u>(20) Multiprocessors, (128) CUDA Cores/MP: </u>	<u>2560 CUDA Cores</u>
GPU Max Clock rate:	1734 MHz (1.73 GHz)
Memory Clock rate:	5005 Mhz
Memory Bus Width:	256-bit
L2 Cache Size:	2097152 bytes
Maximum Texture Dimension Size (x,y,z)	1D=(131072), 2D=(131072, 65536), 3D=(16384, 16384, 16384)
Maximum Layered 1D Texture Size, (num) layers	1D=(32768), 2048 layers
Maximum Layered 2D Texture Size, (num) layers	2D=(32768, 32768), 2048 layers
Total amount of constant memory:	65536 bytes



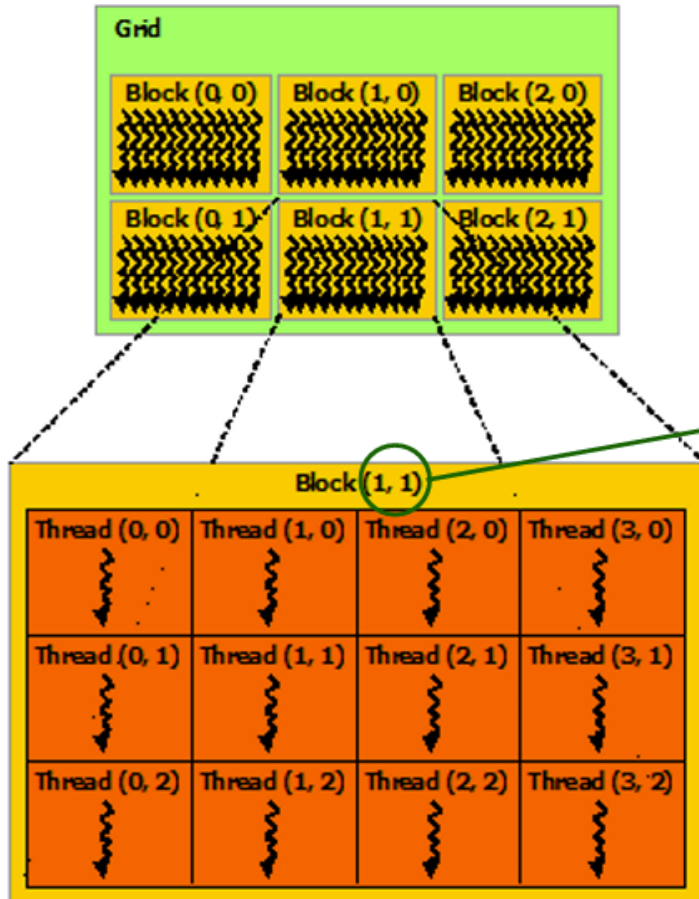
CUDA : Programming

```
Total amount of shared memory per block:      49152 bytes
Total number of registers available per block: 65536
Warp size:                                     32
Maximum number of threads per multiprocessor: 2048
Maximum number of threads per block:           1024
Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
Max dimension size of a grid size (x,y,z): (2147483647, 65535, 65535)
Maximum memory pitch:                          2147483647 bytes
Texture alignment:                             512 bytes
Concurrent copy and kernel execution:          Yes with 2 copy engine(s)
Run time limit on kernels:                     No
Integrated GPU sharing Host Memory:            No
Support host page-locked memory mapping:       Yes
Alignment requirement for Surfaces:            Yes
Device has ECC support:                        Disabled
Device supports Unified Addressing (UVA):       Yes
Supports Cooperative Kernel Launch:            Yes
Supports MultiDevice Co-op Kernel Launch:      Yes
Device PCI Domain ID / Bus ID / location ID:   0 / 65 / 0
Compute Mode:
    < Default (multiple host threads can use ::cudaSetDevice() with device simultaneously) >
```

```
deviceQuery, CUDA Driver = CUDART, CUDA Driver Version = 9.1, CUDA Runtime Version = 9.1, NumDevs = 1
Result = PASS
```



CUDA : Programming



- `numBlocks` organization of the block grid is similar to `threadsPerBlock`
- Can again be one-dimensional, two-dimensional or three-dimensional
- Again `dim3` declaration
- `blockIdx.[x,y,z]` is again a built-in variable at the kernel level to identify corresponding thread blocks for each of the parallel threads

CUDA : Programming

Multiple Thread Blocks Matrix Addition

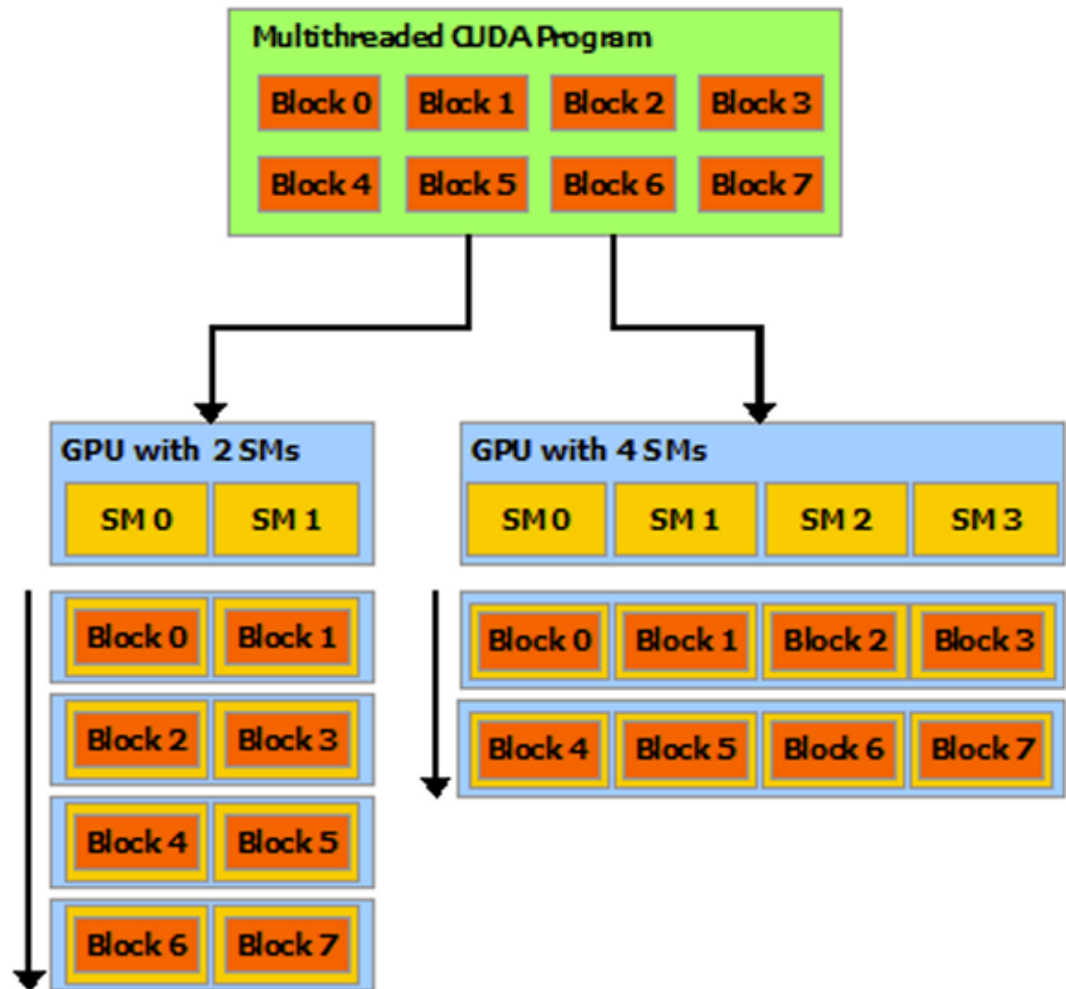
```
#define N 256;

// kernel definition;
__global__ void MatAdd(float **A, float **B, float **C)
{
    int i, j;
    i = (blockIdx.x * blockDim.x) + threadIdx.x;
    j = (blockIdx.y * blockDim.y) + threadIdx.y;
    C[i][j] = A[i][j] + B[i][j];
}

int main()
{
    dim3 threadsPerBlock, numBlocks;
    ...
    // kernel invocation with blocks of 256 threads
    threadsPerBlock.x = 16;
    threadsPerBlock.y = 16;
    numBlocks.x = N / threadsPerBlock.x;
    numBlocks.y = N / threadsPerBlock.y;
    MatAdd <<< numBlocks, threadsPerBlock >>> (A, B, C);
    ...
}
```

CUDA : Programming

- 256 threads per thread block is arbitrary but a frequent choice
- Thread blocks are required to execute independently in any order
- Scalability results from this requirement



CUDA : Wrap-up

- GPU kernels responsible for proper logic
- Execution configuration responsible for efficient employment of GPU resources
- Massive parallelism at the level of GPU threads replacing conventional loops over array elements with many individual threads directly acting on thread-specific elements in parallel
- Built-in variables to quasi-automatize various workloads, e.g. threadIdx, blockIdx 0,1,2,3...
- Shapes of thread blocks go hand in hand with domain decomposition (vector, matrix, volume)
- GPU computing means eco-friendly computing

CUDA : ... and more

- CUDA libraries
 - cuBLAS : linear algebra
 - cuSPARSE: linear algebra for sparse matrices
 - cuSOLVER: abstraction of cuBLAS and cuSPARSE
 - cuFFT: for fast furier transformation
 - cuRAND: for random number generation
 - NPP: 2D image and signal processing
 - cuTENSOR: tensor linear algebra library
 - cuDNN: deep neural network
- Deep learning framework/environment with CUDA back-end enabled
 - pyTorch, tensorflow, keras,...
 - Colab/Kaggle,...